



SEATECH ENGINEERING SCHOOL
Data Science and Intelligent Systems Program

Project Report

Continual Learning

*Exploring Exemplar-Based Continual Learning Strategies for
Image Classification*

Prepared by:

Mohammed ABO ORAIG
Mugahed BIN SNKAR

Supervised by:

Cong Tu TRAN

Academic Year 2025–2026

Contents

1	Problem Definition	2
1.1	Naive Baseline on Split CIFAR100	2
1.2	Naive Baseline on COrE50	3
1.3	Interpretation	3
2	iCaRL: Incremental Classifier and Representation Learning	4
2.1	Exemplar Selection via Herding	4
2.2	Knowledge Distillation	5
2.3	Nearest-Mean-of-Exemplars (NME) Classification	6
2.4	Summary of the iCaRL Algorithm	6
2.5	Strengths and Limitations	6
2.6	iCaRL Performance on CIFAR-100	7
2.7	iCaRL Performance on COrE50	7
2.8	Comparison of iCaRL on CIFAR100 and COrE50	8
3	MIR: Maximally Interfered Retrieval	9
3.1	Interference-Based Sample Selection	9
3.2	Method Description	9
3.3	Summary of the MIR Algorithm	10
3.4	Strengths and Limitations	11
3.5	MIR Performance on CIFAR-100	11
3.6	MIR Performance on COrE50	12
3.7	Comparison of MIR on CIFAR-100 and COrE50	12
4	Conclusion	13

1 Problem Definition

Modern deep learning systems are typically trained under the assumption that all training data is available at once. However, many real-world applications do not satisfy this assumption. In practical scenarios data arrives sequentially over time and models must adapt continuously without retraining from scratch. This setting is known as Continual Learning (CL).

Formally, instead of training on a single dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, the model is exposed to a sequence of datasets (experiences):

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \dots, \mathcal{D}^{(T)}$$

where each $\mathcal{D}^{(t)}$ becomes available only once and past data is no longer accessible.

In the Class-Incremental Learning (CIL) scenario, which is the focus of this work, each experience introduces new classes. After training on all experiences, the model must correctly classify samples among all classes seen so far, without knowing from which experience they originate.

The central challenge in CL is catastrophic forgetting. When a neural network is trained sequentially using standard stochastic gradient descent, the optimization process updates model parameters to fit new data, often overwriting knowledge acquired from previous tasks. As a result, performance on earlier classes can drop dramatically.

Therefore, the objective of CL is to learn a sequence of tasks while maintaining high performance on all previously learned tasks. An effective CL algorithm must balance two conflicting goals:

- Plasticity: the ability to learn new information.
- Stability: the ability to preserve previously learned knowledge.

This stability–plasticity dilemma defines the core difficulty of CL and motivates the development of specialized training strategies.

To illustrate this problem, we first evaluate a Naive baseline, where the model is sequentially trained on each experience without any continual learning mechanism (no replay, no regularization, no memory).

1.1 Naive Baseline on Split CIFAR100

Figure 1 reports the stream accuracy after each experience for the Split CIFAR100 benchmark. The model achieves a high accuracy of 67.3% after the first experience. However, as new experiences are learned, performance rapidly degrades. After the final experience, the accuracy drops to only 7.49%. The final evaluation metrics are:

- Average Accuracy (AA): 0.0749
- Average Incremental Accuracy (AIA): 0.2077
- Forgetting Measure (FM): 0.7163

The high forgetting measure ($FM \approx 0.72$) clearly indicates that most of the knowledge acquired during earlier experiences has been lost. The drastic reduction from 67% to 7% confirms the instability of standard training in a sequential learning scenario.

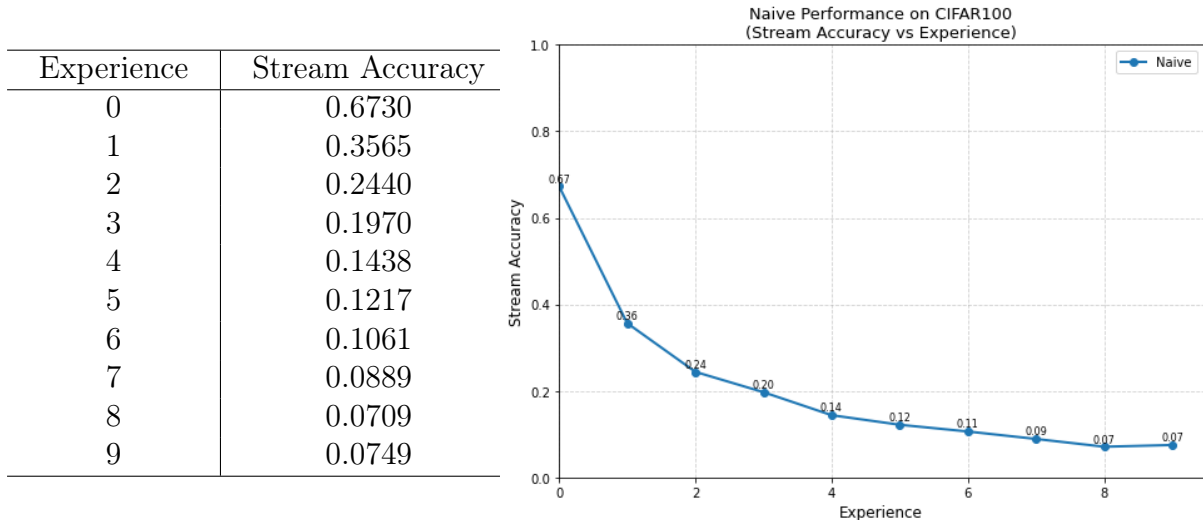


Figure 1: Naive baseline performance on Split CIFAR100.

1.2 Naive Baseline on COrE50

On the COrE50 benchmark (New Classes scenario), the evaluation protocol differs from CIFAR100. In Avalanche, the test stream is composed of a single experience that contains all classes. Consequently, the stream accuracy is computed over the entire set of classes, including both seen and unseen ones, from the very beginning of training.

This explains why the accuracy starts relatively low at the first experience (19.4%). The final evaluation metrics are:

- Average Accuracy (AA): 0.0814
- Average Incremental Accuracy (AIA): 0.0800
- Forgetting Measure (FM): 0.0573

At this stage, the model has been trained only on a small subset of classes, while predictions for unseen classes are essentially random. As new classes are progressively introduced, the model learns to classify them; however, since no rehearsal or regularization mechanism is used in the naive approach, previously learned classes are rapidly forgotten.

As a result, the global stream accuracy remains low throughout the incremental process, fluctuating between 6% and 8% in later experiences. This behavior confirms that catastrophic forgetting is not dataset-specific but rather intrinsic to sequential training without dedicated continual learning mechanisms.

1.3 Interpretation

The naive baseline results clearly illustrate the fundamental challenge of CL.

- Standard supervised training optimizes performance only on the current experience.
- Model parameters progressively adapt to the most recent data distribution.
- Previously learned representations are overwritten due to the absence of memory or regularization mechanisms.

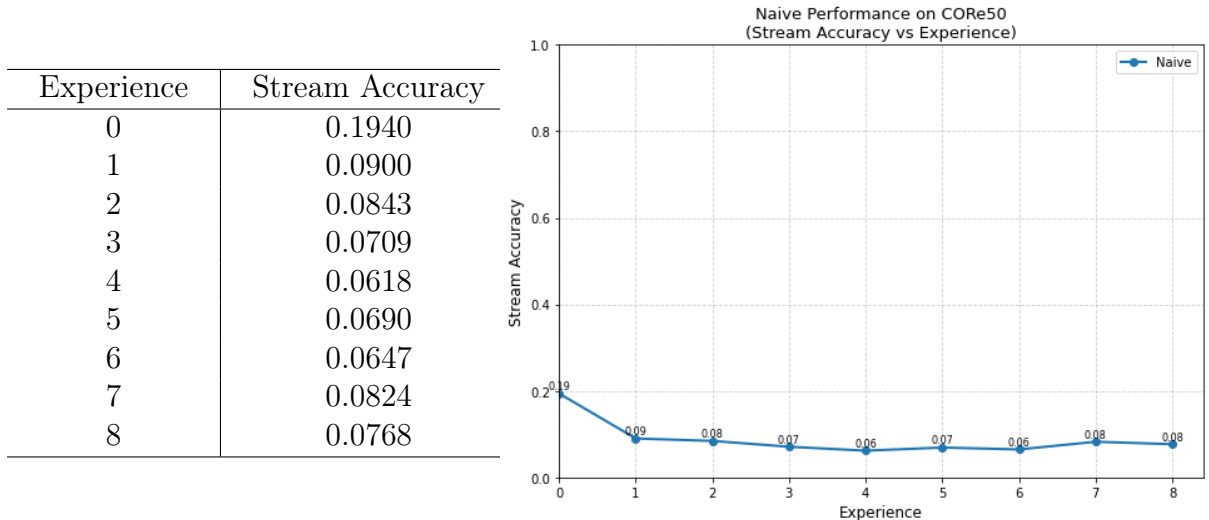


Figure 2: Naive baseline performance on COrE50 (global stream accuracy).

In the COrE50 setting, where evaluation is performed over the full set of classes at every step, this effect becomes even more apparent: as new classes are introduced, performance on earlier ones rapidly deteriorates, leading to persistently low global accuracy.

These observations confirm that catastrophic forgetting is an inherent consequence of sequential optimization. Without explicit mechanisms to preserve past knowledge, the model behaves as if earlier experiences had never been learned.

This severe degradation motivates the use of dedicated continual learning strategies. In the following sections, we investigate two such approaches — iCaRL and MIR — and evaluate their ability to mitigate forgetting under different benchmark settings.

2 iCaRL: Incremental Classifier and Representation Learning

Incremental Classifier and Representation Learning (iCaRL) is an exemplar-based continual learning method specifically designed for the class-incremental learning (CIL) setting. It was introduced to address catastrophic forgetting while maintaining a single shared classifier over all observed classes.

iCaRL is based on three fundamental pillars:

- Exemplar selection via herding,
- Knowledge distillation to prevent forgetting,
- Nearest-Mean-of-Exemplars classification.

These components jointly allow iCaRL to maintain performance on previously learned classes while adapting to new ones.

2.1 Exemplar Selection via Herding

iCaRL maintains a fixed-size memory buffer of exemplars from previously learned classes. Let K denote the total memory budget and t the number of learned classes. Each class

is allocated the following number of exemplars:

$$m = \left\lfloor \frac{K}{t} \right\rfloor$$

To select representative samples, ICaRL uses a procedure called *herding*. The goal is to approximate the true class mean in feature space.

Let $\phi(x)$ be the feature representation extracted by the neural network. For a class c , its true mean in feature space is:

$$\mu_c = \frac{1}{|D_c|} \sum_{x \in D_c} \phi(x)$$

Herding selects exemplars $\{x_1, \dots, x_m\}$ such that their mean:

$$\hat{\mu}_c = \frac{1}{m} \sum_{i=1}^m \phi(x_i)$$

is as close as possible to μ_c .

This greedy selection ensures that the stored exemplars are maximally representative of the original class distribution.

2.2 Knowledge Distillation

When learning new classes, the network parameters are updated. Without constraints, this would alter the representation of old classes, leading to forgetting.

To prevent this, ICaRL uses knowledge distillation. Before training on experience t , a copy of the model is stored as the reference model.

During training, the loss function combines:

- A classification loss on new classes,
- A distillation loss on previously learned classes.

For an input x , let:

$$\mathbf{p}^{old}(x)$$

be the soft targets produced by the previous model, and

$$\mathbf{p}^{new}(x)$$

the outputs of the current model.

The distillation loss enforces:

$$\mathbf{p}^{new}(x) \approx \mathbf{p}^{old}(x)$$

This regularizes the network and stabilizes its predictions for old classes. The overall loss can be written as:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{L}_{dist}$$

where $\mathcal{L}_{CE}$ is the cross-entropy loss for new classes and $\mathcal{L}_{dist}$ preserves knowledge of previous ones.

2.3 Nearest-Mean-of-Exemplars (NME) Classification

Unlike standard classifiers that rely on a learned linear output layer, ICaRL performs inference using a Nearest-Mean-of-Exemplars (NME) rule.

For each class c , the prototype is computed as the mean of its stored exemplars:

$$\hat{\mu}_c = \frac{1}{m} \sum_{x \in M_c} \phi(x)$$

Given a test sample x , classification is performed by selecting the closest prototype in feature space:

$$\hat{y} = \arg \min_c \|\phi(x) - \hat{\mu}_c\|$$

This strategy mitigates bias toward recently learned classes and provides better stability in class-incremental settings.

2.4 Summary of the ICaRL Algorithm

The complete iCaRL procedure is summarized in Algorithm 1.

Algorithm 1 iCaRL Incremental Learning Algorithm

- 1: Initialize feature extractor f_{θ_0}
 - 2: Initialize empty memory $\mathcal{M}_0$
 - 3: **for** each experience $t = 1, \dots, T$ **do**
 - 4: Receive dataset $\mathcal{D}_t$
 - 5: Store previous model $f_{\theta_{t-1}}$
 - 6: Train on $\mathcal{D}_t \cup \mathcal{M}_{t-1}$ using $\mathcal{L}_{CE} + \mathcal{L}_{distill}$
 - 7: Update exemplar memory using herding
 - 8: Recompute class means
 - 9: **end for**
 - 10: Perform inference using nearest-mean classification
-

2.5 Strengths and Limitations

Strengths

- Mitigates catastrophic forgetting through exemplar rehearsal and knowledge distillation.
- Supports continuous representation learning in the class-incremental setting.
- Achieves strong performance under a fixed memory budget.

Limitations

- Requires storing raw exemplars, introducing memory overhead.
- Performance is sensitive to memory size, as exemplars per class decrease over time.
- Relies on greedy herding and nearest-mean classification, which may be suboptimal.

Overall, iCaRL balances stability and plasticity through rehearsal and distillation, making it a foundational baseline in class-incremental learning.

2.6 iCaRL Performance on CIFAR-100

Figure 3 shows the stream accuracy of iCaRL across experiences for different memory sizes on CIFAR-100 benchmark.

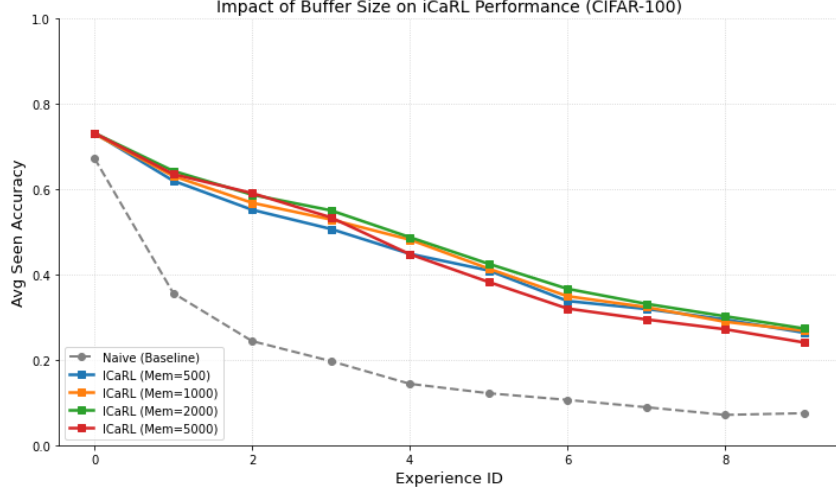


Figure 3: Stream accuracy of iCaRL vs baseline across experiences on CIFAR-100.

After the first experience, iCaRL achieves a high accuracy of approximately 73%. As new experiences are introduced, performance gradually decreases, but the degradation is much slower than for the naive baseline, which suffers severe forgetting (final AA $\approx 7.5\%$). With iCaRL, the final accuracy reaches 26–27%, the average incremental accuracy (AIA) is 45–47%, and the forgetting measure (FM) is reduced by nearly 50% to 0.354–0.406. Compared to the naive baseline, iCaRL effectively mitigates catastrophic forgetting. Increasing memory size provides only modest gains, as the exemplar-per-class ratio becomes very small in the 100-class setting.

2.7 iCaRL Performance on CORE50

Figure 4 shows the stream accuracy of iCaRL across experiences for different memory sizes on the CORE50 benchmark.

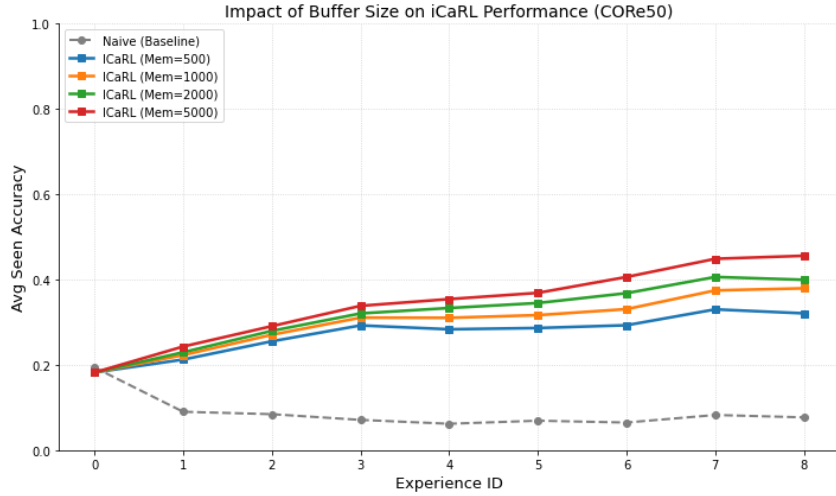


Figure 4: Stream accuracy of iCaRL vs naive baseline across experiences on CORE50.

Unlike CIFAR100, the stream accuracy increases over experiences. This behavior is due to the evaluation protocol: performance is computed over all 50 classes at every step. Early accuracy is low ($\approx 18\%$) because many classes are unseen; as more classes are learned, global accuracy naturally rises.

Final AA improves consistently with memory size (from 32% to 45.5%), and AIA increases accordingly. The forgetting measure becomes negative (-0.13 to -0.27), indicating positive backward transfer rather than degradation. Thus, the rising curve reflects expanding class coverage and representation refinement, not the absence of forgetting.

2.8 Comparison of iCaRL on CIFAR100 and COrE50

For a fair comparison, iCaRL was evaluated on both benchmarks using the same protocol: stream accuracy computed over the full label space at each experience.

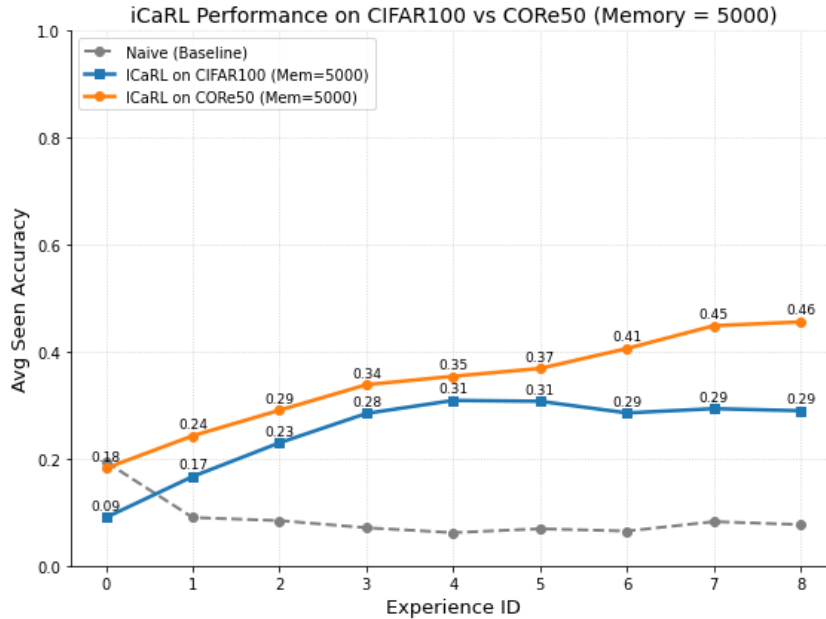


Figure 5: Enter Caption

On CIFAR100, the final AA remains limited (26%–28% depending on memory size), with positive forgetting ($FM \approx 0.42$ – 0.46), indicating that catastrophic forgetting persists despite rehearsal and distillation.

In contrast, COrE50 shows substantially higher performance. AA increases from 32% to 45.5% as memory grows, and the forgetting measure becomes negative (-0.13 to -0.27), reflecting positive backward transfer rather than degradation.

The gap is mainly explained by structural differences between the datasets:

- **Number of classes:** CIFAR100 contains 100 classes, while COrE50 contains 50. With a fixed memory budget, CIFAR100 stores fewer exemplars per class, weakening prototype estimation.
- **Visual complexity:** CIFAR100 consists of low-resolution (32×32) and fine-grained categories with high inter-class similarity, increasing interference. COrE50 objects are more distinct and object-centric, which favors NME classification.

- **Effect of pretraining:** ImageNet pretraining aligns better with CORE50’s visual structure. On CIFAR100, the benefit is limited due to resolution mismatch and dense class overlap.

Overall, iCaRL consistently outperforms naive training on both benchmarks. However, its effectiveness strongly depends on the exemplar-to-class ratio and dataset structure. The method scales better in moderately-sized, well-separated object recognition scenarios (CORE50) than in highly dense and fine-grained settings such as CIFAR100.

3 MIR: Maximally Interfered Retrieval

Maximally Interfered Retrieval (MIR) improves replay efficiency by prioritizing memory samples that are most at risk of being forgotten. Instead of selecting samples randomly, MIR identifies those whose loss would increase the most after learning new data.

The central intuition behind MIR is that not all memory samples are equally important. Some samples are more vulnerable to interference caused by updates on new data. MIR dynamically estimates this interference and focuses replay on the most affected samples.

3.1 Interference-Based Sample Selection

The key idea of MIR is to estimate how much previously learned samples would be negatively affected after learning new data. Samples that experience the largest increase in loss are considered the most vulnerable and are selected for replay.

MIR relies on a fixed-size memory buffer storing samples from previous experiences. The buffer can be managed using reservoir sampling or balanced storage strategies. Even with limited memory, MIR remains effective because it prioritizes the most critical samples rather than replaying randomly selected ones.

3.2 Method Description

Let:

- θ denote the model parameters,
- B be the current mini-batch from experience t ,
- $\mathcal{M}$ be the replay memory buffer,
- α be the learning rate.

The MIR procedure operates as follows.

Step 1: Gradient Computation Compute the gradient of the loss on the incoming mini-batch:

$$g = \nabla_{\theta} L(B, \theta)$$

Step 2: Virtual Parameter Update Apply a temporary (virtual) update:

$$\theta' = \theta - \alpha g$$

This simulates how the parameters would change after learning the new data.

Step 3: Interference Measurement For each memory sample $x_i \in \mathcal{M}$, compute the loss difference:

$$\Delta L_i = L(x_i, \theta') - L(x_i, \theta)$$

A positive ΔL_i indicates that the sample is negatively affected by the update.

Step 4: Selection of Most Interfered Samples Select the top- k samples with the largest interference values:

$$S^* = \text{TopK}_{x_i \in \mathcal{M}}(\Delta L_i)$$

Step 5: Final Corrective Update Perform the final update using both the current mini-batch and the selected memory samples:

$$L_{\text{total}} = L(B \cup S^*, \theta)$$

This corrective update mitigates forgetting by explicitly protecting the most vulnerable samples.

3.3 Summary of the MIR Algorithm

Algorithm 2 MIR for Class-Incremental Learning

- 1: Initialize model parameters θ
 - 2: Initialize replay memory $\mathcal{M} = \emptyset$
 - 3: **for** each experience $t = 1, \dots, T$ **do**
 - 4: Receive dataset D_t
 - 5: **for** each minibatch $B \subset D_t$ **do**
 - 6: Compute gradient $g = \nabla_{\theta} L(B, \theta)$
 - 7: Compute virtual parameters $\theta' = \theta - \alpha g$
 - 8: Compute interference scores ΔL_i for $x_i \in \mathcal{M}$
 - 9: Select S^* as the top- k interfered samples
 - 10: Update θ using $B \cup S^*$
 - 11: Update memory $\mathcal{M}$
 - 12: **end for**
 - 13: **end for**
-

3.4 Strengths and Limitations

Strengths

- More effective than naive random replay in reducing catastrophic forgetting.
- Efficient use of limited memory capacity.
- Easy integration into standard SGD-based training pipelines.
- Applicable in single-head class-incremental settings.

Limitations

- Additional computational overhead due to the virtual update step.
- Performance depends on memory size and hyperparameter tuning.
- May struggle in highly non-stationary or large-scale scenarios.

3.5 MIR Performance on CIFAR-100

Figure 6 presents the performance of MIR on the CIFAR-100 class-incremental benchmark for different memory sizes. CIFAR-100 is a relatively controlled dataset with limited intra-class variability, which makes it more stable for continual learning compared to real-world datasets.

The results show that MIR benefits significantly from larger replay memory. When the memory size is small (e.g., 500 samples), the model suffers from stronger forgetting, resulting in lower overall accuracy. Increasing the memory size to 2000 and 5000 improves stability and allows the model to better retain previously learned knowledge.

Across experiences, the accuracy initially increases as the model accumulates knowledge from multiple classes. After several experiences, a slight degradation is observed due to interference between tasks, which is typical in class-incremental learning scenarios. Nevertheless, MIR maintains relatively stable performance, especially with larger memory buffers.

These results confirm that replay memory plays a critical role in maintaining performance and reducing forgetting in continual learning.

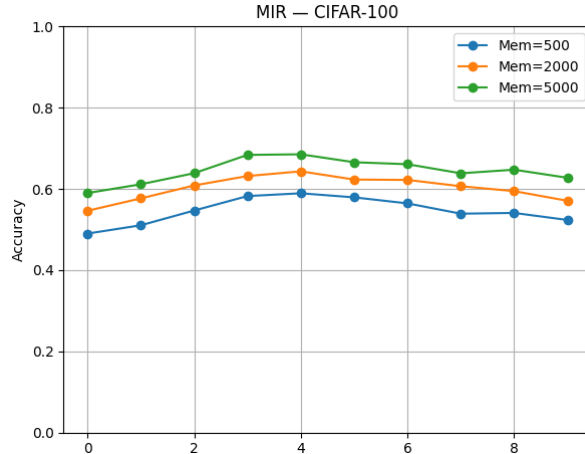


Figure 6: MIR accuracy across experiences on CIFAR-100 for different memory sizes.

3.6 MIR Performance on CORE50

Figure 7 shows the performance of MIR on the CORE50 benchmark. Unlike CIFAR-100, CORE50 is a more challenging dataset composed of real-world object images captured under varying conditions such as illumination, pose, background, and sensor noise. These variations increase task interference and make continual learning more difficult.

Compared to CIFAR-100, overall accuracy is lower across all memory sizes. This behaviour is expected due to the higher complexity and variability of CORE50. Nevertheless, MIR still benefits from larger memory buffers. Increasing the memory size from 500 to 5000 leads to noticeable improvements in both stability and final accuracy.

The accuracy typically increases during the first few experiences as the model learns new object categories, then slightly decreases due to interference from subsequent experiences. This pattern reflects the inherent difficulty of class-incremental learning in real-world scenarios.

Overall, MIR demonstrates the ability to mitigate forgetting even in challenging conditions, although performance remains sensitive to dataset complexity and memory capacity.

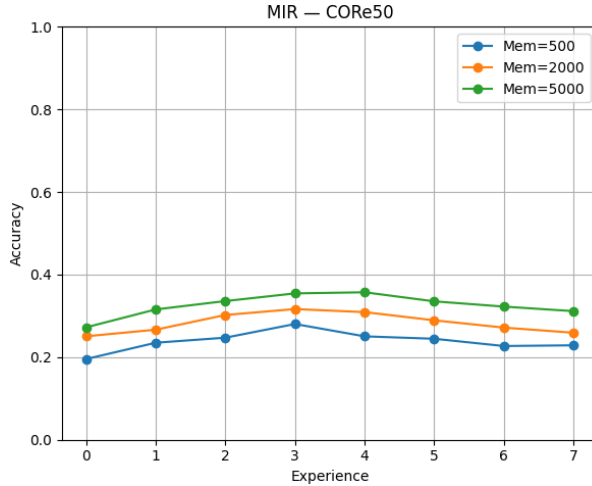


Figure 7: MIR accuracy across experiences on CORE50 for different memory sizes.

3.7 Comparison of MIR on CIFAR-100 and CORE50

We compare the performance of MIR on two class-incremental benchmarks: CIFAR-100 and CORE50. The results show that MIR achieves higher accuracy and more stable learning on CIFAR-100 than on CORE50.

CIFAR-100 contains clean and structured images with lower intra-class variability, which reduces task interference and allows the model to better retain previously learned knowledge. In contrast, CORE50 is a more challenging real-world dataset with variations in illumination, pose, background, and noise, leading to stronger interference and increased forgetting.

Across both datasets, increasing the memory buffer improves performance and reduces forgetting. However, dataset complexity remains a key factor: MIR performs more effectively on CIFAR-100, while continual learning on CORE50 remains more difficult due to higher variability and task interference.

4 Conclusion

In this project, we investigated the problem of Continual Learning under the class-incremental setting, where a model must learn sequentially from a stream of data while preserving previously acquired knowledge. The naive baseline clearly demonstrated the severity of catastrophic forgetting, with performance rapidly degrading as new experiences were learned.

To address this issue, we studied two replay-based methods: iCaRL and MIR. Both approaches rely on a limited memory buffer to preserve past knowledge, but they differ in how replay samples are selected and utilized.

The iCaRL method combines exemplar rehearsal, knowledge distillation, and nearest-mean classification. It provides stable performance and effectively reduces forgetting, especially when memory is sufficient. However, its performance depends on the number of stored exemplars per class and can degrade in large-scale scenarios with many classes.

The MIR method improves replay efficiency by prioritizing memory samples that are most vulnerable to interference. By selecting samples whose loss increases the most after learning new data, MIR provides targeted protection against forgetting. Experimental results showed that MIR consistently outperforms naive training and benefits from larger memory buffers, although performance remains sensitive to dataset complexity.

Across both methods, experiments on CIFAR-100 and CORe50 highlighted the impact of dataset characteristics on continual learning. CIFAR-100, being more structured and less noisy, allows more stable learning and higher accuracy, while CORe50 presents a more challenging real-world scenario with stronger interference and variability.

Overall, this study confirms that replay-based strategies are effective for mitigating catastrophic forgetting, but their performance strongly depends on memory capacity, training configuration, and dataset complexity. Future work may explore hybrid approaches combining replay with regularization, adaptive memory management strategies, and more scalable continual learning methods.

References

- [1] ContinualAI, “Avalanche: An End-to-End Library for Continual Learning,” Available: <https://avalanche.continualai.org/>
- [2] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Technical Report, University of Toronto, 2009. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [3] V. Lomonaco and D. Maltoni, “CRe50: A New Dataset and Benchmark for Continuous Object Recognition,” Available: <https://vlomonaco.github.io/core50/>
- [4] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “iCaRL: Incremental Classifier and Representation Learning,” in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2001–2010.
- [5] R. Aljundi, E. Belilovsky, T. Tuytelaars, L. Charlin, M. Caccia, M. Lin, and L. Page-Caccia, “Online Continual Learning with Maximal Interfered Retrieval,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 11849–11860.
- [6] L. Wang, X. Zhang, H. Su, and J. Zhu, “A Comprehensive Survey of Continual Learning: Theory, Method and Application,” IEEE.
- [7] Y. Wang and Y. Huang, “Reduced Memory Size A-GEM With CGAN,” Department of Electrical and Computer Engineering, New York University, Tandon of Engineering.
- [8] A. Dutt, *Continual Learning for Image Classification*, PhD thesis, Université Grenoble Alpes, 2019.
- [9] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, “Efficient Lifelong Learning with A-GEM,” University of Oxford and Facebook AI Research.
- [10] R. Aljundi et al., “Online Continual Learning with Maximal Interfered Retrieval,” in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pp. 11849–11860, 2019.